

10장 라이브니스 분석 (Liveness Analysis)

최광훈
전남대학교

개요

컴파일러의 프런트엔드에서 프로그램을 중간 언어로 번역할 때 사용되는 임시 변수의 수에는 제한이 없지만, 이 프로그램은 제한된 수의 레지스터를 가진 기계에서 실행됨

임시 변수 **a**와 **b**가 동시에 사용 중이 아니라면, **a**와 **b**는 같은 레지스터에 저장될 수 있음

많은 임시 변수들이 적은 수의 레지스터에 들어갈 수 있음. 만약 모두 들어가지 못한다면, 초과되는 임시 변수들은 메모리에 보관할 수 있음.

따라서, 컴파일러는 중간 표현 프로그램을 분석하여 어떤 임시 변수들이 동시에 사용 중인지 판단해야 함

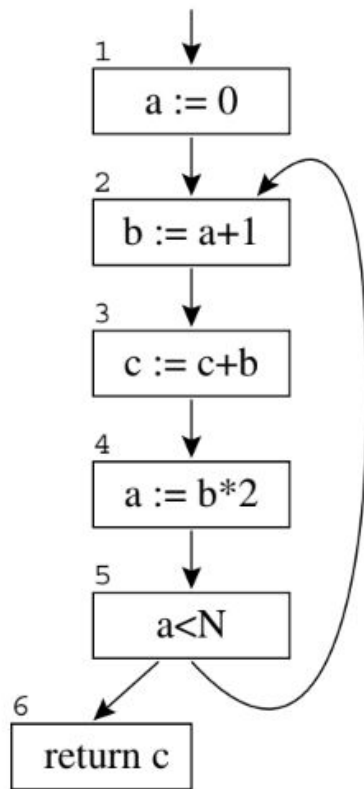
- 어떤 변수가 미래에 필요할 수 있는 값을 가지고 있다면, 그 변수는 라이브(**live**) 상태인

제어 흐름 그래프(Control-flow graph)

(1) 문장 $\Rightarrow$ 노드,

(2) 문장 x ; 문장 $y \Rightarrow$ 에지

$a \leftarrow 0$
 $L_1 : b \leftarrow a + 1$
 $c \leftarrow c + b$
 $a \leftarrow b * 2$
if $a < N$ goto L_1
return c



라이브니스 분석(liveness analysis)

변수의 값은 미래에 사용된다면 라이브(live)함

뒤에서 앞으로 거슬러 올라오며 라이브니스 분석 수행

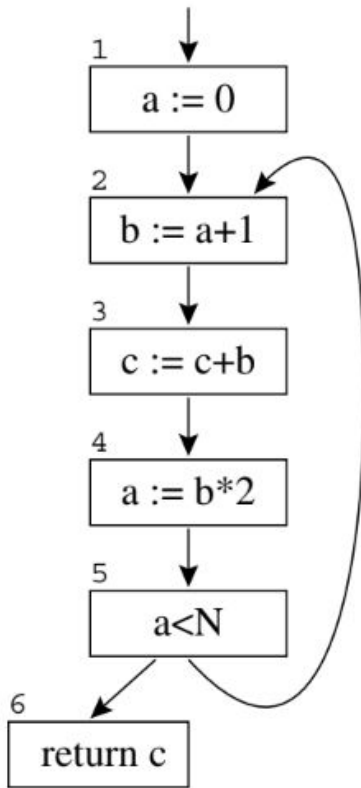
- 변수 **b**의 live range: 2번 -> 3번 -> 4번 (1번에서 **b**에 할당)

- 변수 **a**의 live range: 1번 -> 2번, 4번 -> 5번 -> 2번

cf. 2번 -> 3번 -> 4번에서 변수 **a**는 live 하지 않음

(변수 **a**의 값이 정의되어 있지만 사용하지 않음)

- 변수 **c**의 live range: 1번 -> ... -> 6번 (가정:1에서 **c**가 라이브



라이브니스 분석(liveness analysis)

변수 **a**, **b**, **c**를 레지스터 **r1**, **r2**로 저장 가능.

왜냐하면, 변수 **a**와 **b**는 겹치지 않음

- 변수 **a**: 1번 -> 2번, 4번 -> 5번 -> 2번

- 변수 **b**: 2번 -> 3번 -> 4번

- 변수 **c**: 1번 -> ... -> 6번

// r2 (c의 값)

(1) $r1 := 0$

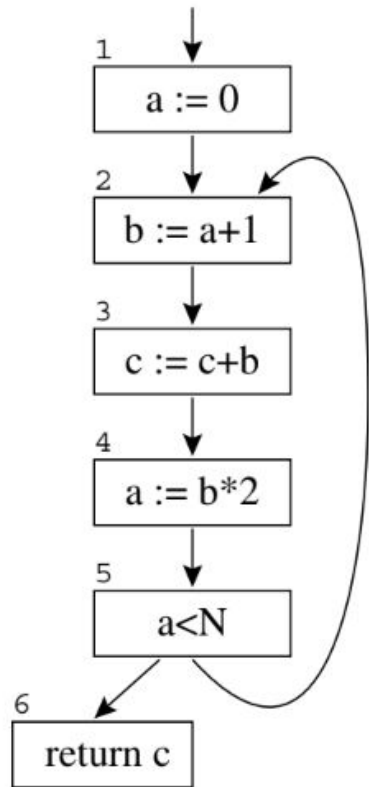
(2) $r1 := r1 + 1$

(3) $r2 := r2 + r1$

(4) $r1 := r1 * 2$

(5) $r1 < N$

(6) return r2



라이브니스 분석(liveness analysis)

변수의 라이브니스는 제어 흐름 그래프의 간선을 따라 ‘흘러가는’ 것으로 볼 수 있고, 각 변수의 라이브 범위를 결정하는 것은 데이터 흐름 문제(dataflow problem)의 한 예임.

Uses & Defs

- $\text{def}(n)$: 문장 n 에서 정의하는 변수 집합
- $\text{use}(n)$: 문장 n 에서 사용하는 변수 집합

	$\text{def}(n)$	$\text{use}(n)$
(1) $a := 0$	{ a }	{ }
(2) $b := a + 1$	{ b }	{ a }
(3) $c := c + b$	{ c }	{ b, c }
(4) $a = b * 2$	{ a }	{ b }
(5) $a < N$ (6), (2)	{ }	{ a }
(6) return c	{ }	{ c }

라이브니스 분석(liveness analysis)

라이브니스 정보 (live-in과 live-out)

in[n] & out[n]

- $\text{in}(n) : \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$

- $\text{use}(n) : \bigcup_{s \in \text{succ}[n]} \text{in}[s]$

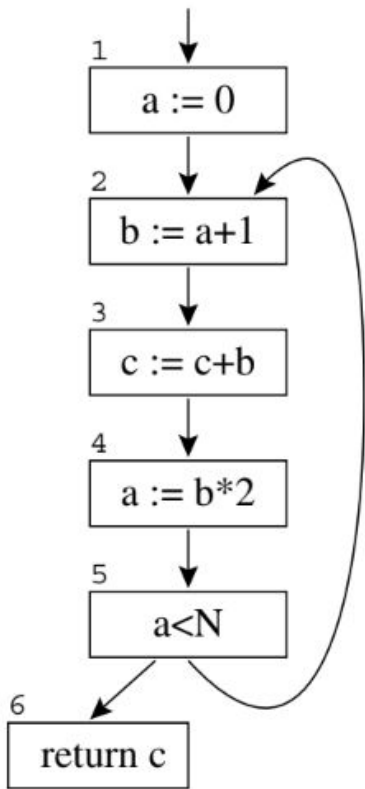
계산 방법(교재 참고):

- 고정점 계산 (fixed point)

- repeat until no change 알고리즘

	in(n)	out(n)
(1) a := 0	{ c }	{ a, c }
(2) b := a + 1	{ a, c }	{ b, c }
(3) c := c + b	{ b, c }	{ b, c }
(4) a = b * 2	{ b, c }	{ a, c }
(5) a < N (6), (2)	{ a, c }	{ a, c }
(6) return c	{ c }	{ }

라이브니스 분석(liveness analysis)

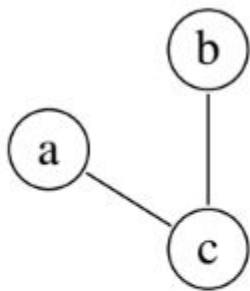


	in(n)	out(n)
(1) a:= 0	{ c }	{ a, c }
(2) b:= a + 1	{ a, c }	{ b, c }
(3) c := c+ b	{ b, c }	{ b, c }
(4) a: = b * 2	{ b, c }	{ a, c }
(5) a < N (6), (2)	{ a, c }	{ a, c }
(6) return c	{ c }	{ }

간섭 그래프(Interference graph)

임시 변수 **a**, **b**가 같은 레지스터에 할당되는 것을 막는 조건을 간섭(interference)이라 함

어떤 지점에서 **a**와 **b**가 동시에 라이브 상태이면 이 둘은 같은 레지스터에 넣을 수 없음

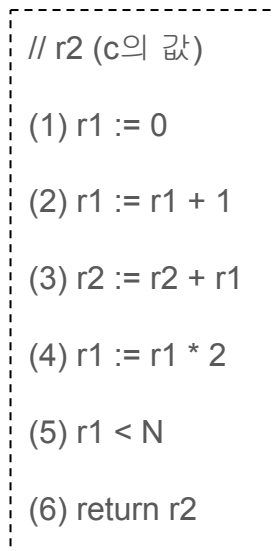
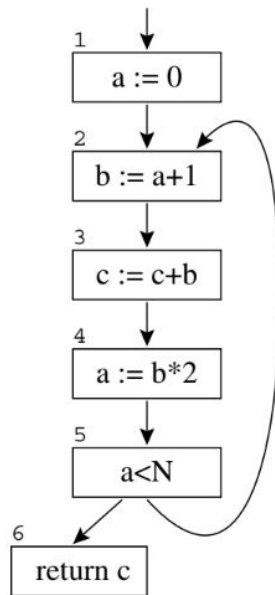
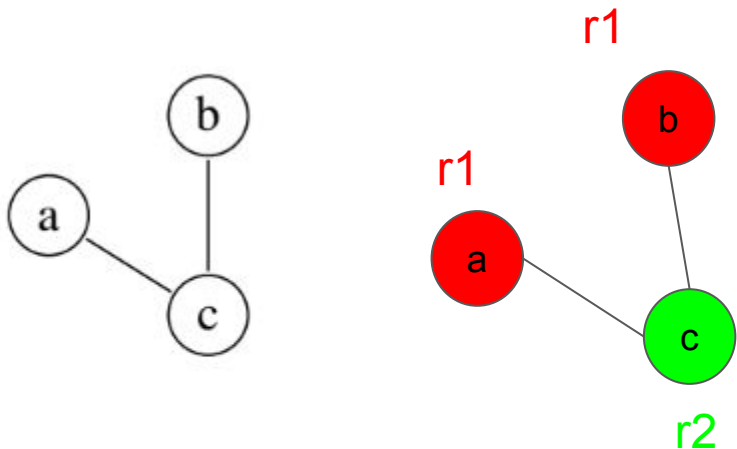


	in(n)	out(n)
(1) $a := 0$	{ c }	{ a, c }
(2) $b := a + 1$	{ a, c }	{ b, c }
(3) $c := c + b$	{ b, c }	{ b, c }
(4) $a = b * 2$	{ b, c }	{ a, c }
(5) $a < N$ (6), (2)	{ a, c }	{ a, c }
(6) return c	{ c }	{ }

간섭 그래프(Interference graph)

레지스터 할당 문제: 그래프 색칠하기 문제

- 간섭 그래프의 노드 (임시 변수) a, b, c 에 색을 칠하되 인접한 노드는 다른 색으로
- 최소 색상의 수 $\Rightarrow$ 최소 레지스터의 갯수



MiniJava 컴파일러의 라이브니스 분석

MiniJava 컴파일러의 두 단계 흐름 분석

- 먼저, **Assem** 프로그램을 분석하여 제어 흐름 그래프(control-flow graph) 를 생성
- 그다음, 이 제어 흐름 그래프에서 변수들의 라이브니스를 분석하여 간섭 그래프(interference graph) 를 생성

MiniJava 컴파일러의 라이브니스 분석

추상 그래프 클래스

- 제어 흐름 그래프와
- 간접 그래프를 표현

```
package Graph;

public class Graph {
    public Graph();
    public NodeList nodes();
    public Node newNode();
    public void addEdge(Node from, Node to);
    public void rmEdge(Node from, Node to);
    public void show(java.io.PrintStream out);
}

public class Node {
    public Node(Graph g);
    public NodeList succ();
    public NodeList pred();
    public NodeList adj();
    public int outDegree();
    public int inDegree();
    public int degree();
    public boolean goesTo(Node n);
    public boolean comesFrom(Node n);
    public boolean adj(Node n);
    public String toString();
}
```

MiniJava 컴파일러의 라이브니스 분석

제어 흐름 그래프

```
public abstract class FlowGraph extends Graph.Graph {  
    public abstract TempList def(Node node);  
    public abstract TempList use(Node node);  
    public abstract boolean isMove(Node node);  
    public void show(java.io.PrintStream out);  
}
```

```
package FlowGraph;  
public class AssemFlowGraph extends FlowGraph {  
    public Instr instr(Node n);  
    public AssemFlowGraph(Assem.InstrList instrs);  
}
```

MiniJava 컴파일러의 라이브니스 분석

라이브니스 분석

```
package RegAlloc;
abstract public class InterferenceGraph extends Graph.Graph{
    abstract public Graph.Node tnode(Temp.Temp temp);
    abstract public Temp.Temp gtemp(Node node);
    abstract public MoveList moves();
    public int spillCost(Node node);
}
```

```
package RegAlloc;
public class Liveness extends InterferenceGraph {
    public Liveness(FlowGraph flow);
}
```

[실습]

1. `AssemFlowGraph` 클래스를 구현하여 `Assem` 명령어 리스트에서 제어 흐름 그래프를 만드시오

- chap10에서 추상 클래스 `Graph.Graph`와 `FlowGraph.FlowGraph` 제공

2. `Liveness` 클래스를 구현하시오.